

SQL ASSIGNMENT

Objective

Apply the SQL concepts learned during the session to clean, transform, and query real-world style data using basic SQL operations like filtering, JOINS, aggregations, and case transformations.

Task DescriptiON

Importing Database Classic Models

ClassicModels is a MYSQL sample database which can be downloaded freely from the internet or your lab instructor will provide you .sql dump database. Procedure to Import database from .sql file

Login to your mysql server in command prompt

```
C:\.....\bin> mysql -u root -p;
```

```
mysql>source sampledatabase.sql;
```

If the .sql file is in other locatiON than the mysql/bin directory then you have to give full path of the file as shown below:

```
mysql> source F:/db/sampledatabase.sql;
```

About MySQL Sample Database: Classic Model

The Classic Models database is a retailer of scale models of classic cars. The sample database contains typical business data such as customers, products, sale orders, sale order line items, etc.

Sample Database Schema

The sample database schema consists of several tables as below:

Customers: Stores customers's data

Products: Stores a list of scale model cars.

ProductLines: Stores a list of product line categories.

Orders: Stores orders placed by customers.

OrderDetails: Stores order line items in each order.

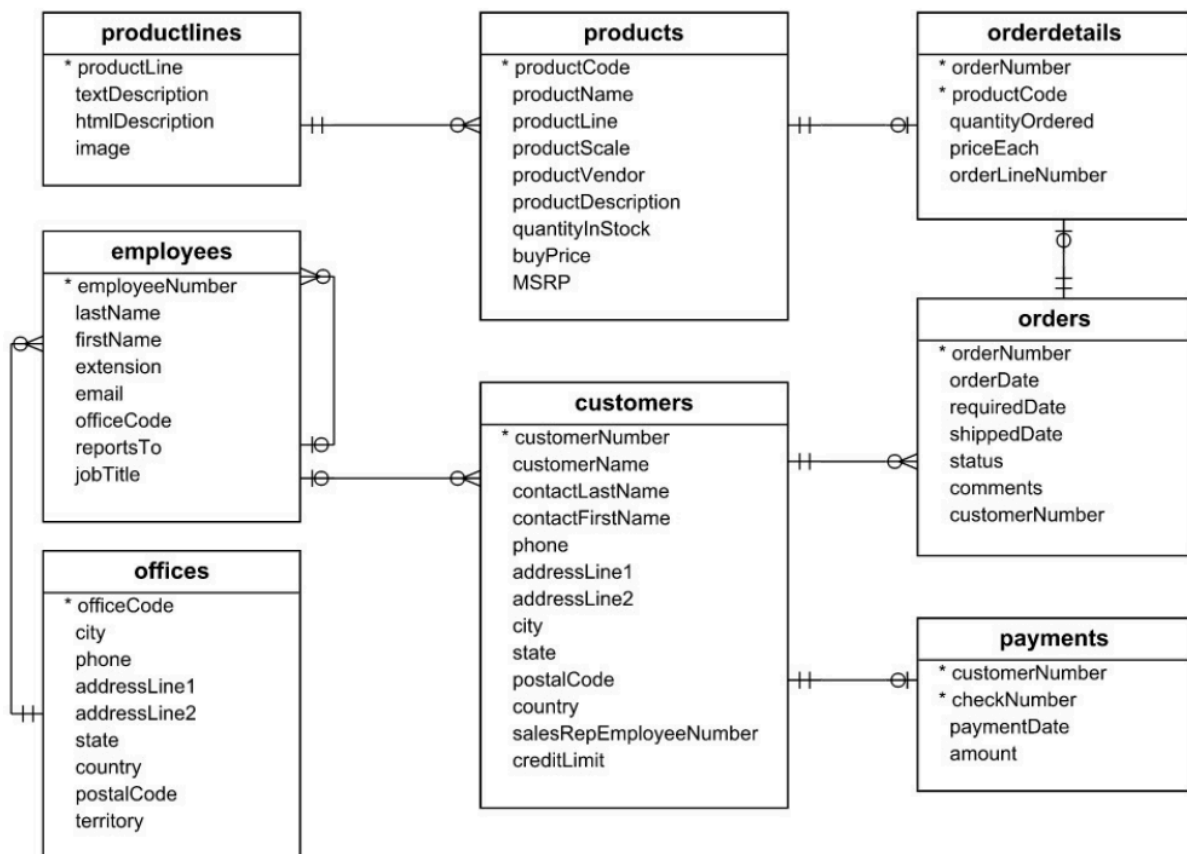
Payments: Stores payments made by customers based ON their account.

Employees: Stores all employee information including organization unit structure such as who reports to whom.

Offices: Stores sell office data.

The ER diagram of the database is as follows:

MySQL Sample Database Diagram



Task:**1. Show all the customers whose creditLimit is greater than 20000**

```
SELECT customerName
FROM customers
WHERE creditLimit > 20000;
```

Simple WHERE clause filtering on a numeric column. The > operator returns only rows where creditLimit is strictly greater than 20000.

2. Show the employees who report to VP Sales.

```
SELECT emp1.employeeNumber, emp1.firstName, emp1.lastName
FROM employees AS emp1
JOIN (
    SELECT *
    FROM employees
    WHERE jobTitle = "VP Sales"
) AS emp2
ON emp1.reportsTo = emp2.employeeNumber;
```

A subquery filters to only the VP Sales record. The outer query then joins employees back to this subquery on the reportsTo → employeeNumber relationship, effectively fetching all direct reports of the VP Sales.

3. Find all the customers who have SET their state while filling the forms and Lives in USA and credit limit is between 100000 and 200000.

```
SELECT customerName, contactFirstName,
       contactLastName, addressLine1,
       addressLine2, city
FROM customers
WHERE state IS NOT NULL
      AND country = "USA"
      AND creditLimit BETWEEN 100000 AND 200000;
```

Combines IS NOT NULL (checks state was provided), equality filter on country, and the BETWEEN operator (inclusive on both ends) for a range filter on creditLimit.

4. Find all the employees who report to Sales Managers of all types.

```
SELECT emp1.employeeNumber, emp1.firstName, emp1.lastName
FROM employees AS emp1
JOIN (
    SELECT *
    FROM employees
    WHERE jobTitle LIKE "Sale% Manager%"
) AS emp2
ON emp1.reportsTo = emp2.employeeNumber;
```

The LIKE operator with % wildcards matches any job title beginning with 'Sale' and containing 'Manager' anywhere after — e.g., 'Sales Manager', 'Sales Manager EMEA'. This makes the query flexible for any regional or typed sales manager title.

5. Find the average credit limit of customers of each country.

```
SELECT country, AVG(creditLimit) AS Avg_CreditLimit
FROM customers
GROUP BY country;
```

GROUP BY aggregates rows per country. AVG() computes the mean creditLimit within each group. The alias Avg_CreditLimit improves readability in the result set.

6. Find the total no. of orders for each date and customer. Show only dates with total number of orders greater than 10 for date and customer.

```
SELECT orderDate, customerNumber, COUNT(*) AS TotalOrders
FROM orders
GROUP BY orderDate, customerNumber
HAVING COUNT(*) > 10;
```

HAVING filters grouped results after aggregation (unlike WHERE which filters before). Grouping by both orderDate and customerNumber creates one row per unique date-customer pair. The result of 0 rows indicates that no single customer placed more than 10 orders on the same date, a realistic finding for this dataset.

7. Find the name of the supervisor, job title of supervisor and total no. of supervisee using subquery. (With out using Join operation)

```
SELECT emp1.firstName, emp1.lastName, emp1.jobTitle,  
      ( SELECT COUNT(*)  
        FROM employees emp2  
        WHERE emp2.reportsTo = emp1.employeeNumber  
      ) AS Supervisee_Count  
FROM employees AS emp1  
WHERE emp1.employeeNumber IN (  
      SELECT DISTINCT reportsTo  
      FROM employees  
      WHERE reportsTo IS NOT NULL  
);
```

A correlated subquery in the SELECT clause counts subordinates for each supervisor on a per-row basis. The WHERE ... IN (subquery) limits results to only employees who appear as a reportsTo target, i.e., actual supervisors. No JOIN is used.

8. Find the name of the supervisor, job title of supervisor and total no. of supervisee using subquery. (With using Join operation)

```
SELECT emp1.firstName, emp1.lastName, emp1.jobTitle, emp2.supervisee_count  
FROM employees emp1  
JOIN (  
      SELECT reportsTo, COUNT(*) AS supervisee_Count  
      FROM employees  
      GROUP BY reportsTo  
    ) AS emp2  
ON emp1.employeeNumber = emp2.reportsTo;
```

The derived table pre-aggregates supervisee counts grouped by reportsTo. This is then joined to the employees table on employeeNumber. JOIN-based aggregation is typically more performant than correlated subqueries because it avoids repeated execution per outer row. Both queries produce identical results.

9. Find all customers with a credit limit greater than average credit credit limit using WITH Clause.

```
WITH avg_credit AS (  
    SELECT AVG(creditLimit) AS avgLimit  
    FROM customers  
)  
SELECT customerNumber, customerName  
FROM customers  
JOIN avg_credit  
    ON customers.creditLimit > avg_credit.avgLimit;
```

The WITH clause (CTE) computes the average credit limit once and stores it as a named temporary result set. The main query then performs a cross-join condition to filter customers above that average. CTEs improve readability compared to inline subqueries and can be referenced multiple times.

10. Find the rank of customer. [Customer with highest credit limit have 1 rank and Customer with lowest credit limit have highest rank]. Then, find the customer with the third highest credit limit.

```
WITH ranked_customers AS (  
    SELECT customerName, creditLimit,  
        RANK() OVER (ORDER BY creditLimit DESC) AS rn  
    FROM customers  
)  
SELECT *  
FROM ranked_customers  
WHERE rn = 3;
```

RANK() is a window function that assigns rank numbers without removing rows. OVER (ORDER BY creditLimit DESC) defines the ranking order. Unlike ROW_NUMBER(), RANK() assigns the same rank to ties and skips subsequent ranks. Filtering rn = 3 retrieves the third-highest distinct rank.

11. Generate a report that shows total no. of employees working in each office.

```
SELECT o.officeCode, o.city, o.state, o.country, COALESCE(e.totalEmployees, 0)
FROM offices o
LEFT JOIN (
    SELECT COUNT(*) AS totalEmployees, officeCode
    FROM employees
    GROUP BY officeCode
) AS e
ON o.officeCode = e.officeCode;
```

LEFT JOIN ensures all offices appear even if they have zero employees. COALESCE(e.totalEmployees, 0) replaces NULL (for offices with no employees in the subquery) with 0. The subquery pre-aggregates employee counts per office.

12. Generate a report that shows total no. of customers visited each office.

```
SELECT o.officeCode, o.city, o.state, o.country,
    COALESCE(COUNT(c.customerNumber), 0) AS totalCustomers
FROM offices o
LEFT JOIN employees e
    ON o.officeCode = e.officeCode
LEFT JOIN customers c
    ON e.employeeNumber = c.salesRepEmployeeNumber
GROUP BY o.officeCode, o.city, o.state, o.country;
```

A chain of two LEFT JOINs links offices → employees → customers via salesRepEmployeeNumber. This three-table join traverses the org hierarchy to attribute each customer to an office.

- 13. Generate a report that shows total payment received by each office using payment tables and essential tables. The report should show the office name, state and country, along with total payments made.**

```
SELECT o.officeCode, o.city, o.state, o.country,  
COALESCE(SUM(p.amount), 0) AS totalPayments  
FROM offices o  
LEFT JOIN employees e  
ON o.officeCode = e.officeCode  
LEFT JOIN customers c  
ON e.employeeNumber = c.salesRepEmployeeNumber  
LEFT JOIN payments p  
ON c.customerNumber = p.customerNumber  
GROUP BY o.officeCode, o.city, o.state, o.country;
```

Four table LEFT JOIN chain (offices → employees → customers → payments).
SUM(p.amount) aggregates all payments attributed to each office's customers.

- 14. Generate a report that shows total sales(in amount) by each office using order details table and other essential tables.**

```
SELECT o.officeCode, o.city, o.state, o.country,  
COALESCE(SUM(od.priceEach * od.quantityOrdered), 0) AS totalSales  
FROM offices o  
LEFT JOIN employees e  
ON o.officeCode = e.officeCode  
LEFT JOIN customers c  
ON e.employeeNumber = c.salesRepEmployeeNumber  
LEFT JOIN orders ord  
ON c.customerNumber = ord.customerNumber  
LEFT JOIN orderdetails od  
ON ord.orderNumber = od.orderNumber  
GROUP BY  
o.officeCode, o.city, o.state, o.country;
```


Five table joins: offices → employees → customers → orders → orderdetails. The computed column `priceEach * quantityOrdered` calculates revenue per line item; `SUM()` aggregates across all line items per office.

15. Generate a report that shows total payment pending for each office.

```
SELECT o.officeCode, o.city, o.state, o.country,
       COALESCE(sales.totalSales, 0) - COALESCE(pays.totalPayments, 0) AS
pendingPayments
FROM offices o
LEFT JOIN (
  SELECT e.officeCode, SUM(od.priceEach * od.quantityOrdered) AS totalSales
  FROM employees e
  LEFT JOIN customers c
    ON e.employeeNumber = c.salesRepEmployeeNumber
  LEFT JOIN orders ord
    ON c.customerNumber = ord.customerNumber
  LEFT JOIN orderdetails od
    ON ord.orderNumber = od.orderNumber
  GROUP BY e.officeCode
) sales
  ON o.officeCode = sales.officeCode
LEFT JOIN (
  SELECT e.officeCode, SUM(p.amount) AS totalPayments
  FROM employees e
  LEFT JOIN customers c
    ON e.employeeNumber = c.salesRepEmployeeNumber
  LEFT JOIN payments p
    ON c.customerNumber = p.customerNumber
  GROUP BY e.officeCode
) pays
  ON o.officeCode = pays.officeCode;
```

Two independent derived tables pre-compute totalSales and totalPayments per office. These are then joined to the offices table and subtracted. COALESCE(..., 0) prevents NULL arithmetic errors for offices with no sales or payments.

16. Find the creditLimit of each person, proportion of creditLimit of each person in each country. [Proportion of person in USA = creditLimit of that person / sum(creditLimit of all person in USA)]

```
SELECT c.customerName, c.country, c.creditLimit,  
       c.creditLimit / NULLIF(cs.creditSum, 0) AS creditProportion  
FROM customers c  
LEFT JOIN (  
    SELECT SUM(creditLimit) AS creditSum, country  
    FROM customers  
    GROUP BY country  
) cs  
ON c.country = cs.country;
```

The derived table computes the total creditLimit per country. NULLIF(cs.creditSum, 0) prevents division-by-zero errors (returns NULL instead of error if sum is 0). Each customer's proportion = their creditLimit / country total. A proportion of 1.0 means that the customer is the only one in their country.

17. Create a view showing the customer name, complete address, and their total number of orders.

```
CREATE VIEW customer_order_summary AS  
SELECT c.customerName, c.addressLine1 AS address1,  
       c.addressLine2 AS address2, c.city, c.state, c.country,  
       COALESCE(COUNT(o.orderNumber), 0) AS totalOrders  
FROM customers c  
LEFT JOIN orders o  
ON c.customerNumber = o.customerNumber  
GROUP BY c.customerName, c.addressLine1, c.addressLine2, c.city, c.state,  
c.country, c.customerNumber;
```

A VIEW stores the query definition, not the data itself — it executes fresh each time it is queried. LEFT JOIN ensures customers with zero orders still appear (122 rows vs. 98 with INNER JOIN). The 24-row difference represents customers who have never placed an order, an important business insight. customerNumber is included in GROUP BY to handle customers with identical names.

18. Update the country of a customer (use any one record).

```
UPDATE customers  
SET country = "Nepal"  
WHERE customerNumber = 496;
```

DML UPDATE statement modifies an existing row. The WHERE clause restricts the update to exactly one record identified by primary key (customerNumber = 496). Without WHERE, all rows would be updated. This change was used to test the country field modification functionality.

19. Delete all payments below 20,000.

```
DELETE FROM payments  
WHERE amount < 20000;
```

DML DELETE removes rows permanently from the table. The WHERE clause filters only small payments for deletion. This simulates a data cleanup operation; e.g., removing test transactions or below-threshold records. Always back up or use transactions before running DELETE in production.

20. Add new payments manually for an existing customer.

```
INSERT INTO payments (customerNumber, checkNumber, paymentDate,  
amount)  
VALUES  
(495, "EU5420759", "2026-05-03", 54202.01),  
(496, "MH97039", "2026-04-05", 38759.30);
```

DML INSERT adds new rows to a table. Multiple rows can be inserted in a single statement using comma separated value tuples, more efficient than multiple single row inserts. The composite primary key of payments is (customerNumber, checkNumber), so each check number must be unique per customer. paymentDate uses the standard ISO format 'YYYY-MM-DD'.